

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)

Activity Tool: Industry Problem Solving Task (Medium)

Assessment Tool Weightage: 35%

Code of Conduct:

I SANTHOSH E (192511470) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E.Santhosh

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Q1. E-commerce company stores 10 million product records

Recommended strategy: Use a sorted/clustered file organization based on Product_ID with a B+ Tree index. Additional indexes can be created on frequently searched fields such as Category_ID and Brand.

Why B+ Tree?

- Supports fast product searches.
- Supports range queries such as price or product-ID ranges.
- Provides efficient insertion and deletion.
- Keeps the index balanced.

Cost analysis:

For a B+ Tree, search cost is $O(\log_B N)$, where N is the number of records and B is the fan-out.

For $N = 10,000,000$ and an approximate fan-out of 100:

$\log_{100}(10,000,000) \approx 4$ levels.

Comparison:

Heap: Search $O(N)$, insertion very fast, poor range queries.

Sorted: Search $O(\log N)$, insertion more expensive, excellent range queries.

Hash: Average search $O(1)$, fast equality search, poor range queries.

B+ Tree: Search $O(\log N)$, efficient updates, excellent range queries.

Conclusion: A B+ Tree indexed sorted/clustered organization is recommended because it provides fast search, efficient range retrieval and good scalability for 10 million records.

Q2. Banking system requires fast retrieval by account number and date

Use a composite B+ Tree index on Account_No and Transaction_Date.

SQL:

```
CREATE INDEX idx_transaction  
ON Transaction(Account_No, Transaction_Date);
```

Example query:

```
SELECT *  
FROM Transaction  
WHERE Account_No = 1001  
AND Transaction_Date BETWEEN '2026-01-01' AND '2026-01-31';
```

The index first locates Account_No = 1001 and then searches the required date range.

Conceptual structure:

B+ Tree

|

Account_No

/ \

1001 1002

|

Transaction_Date
 / | \
 Jan 1 Jan 15 Jan 30

Advantages:

- Fast account-based retrieval.
- Efficient date-range searches for an account.
- Reduces disk I/O.
- B+ Tree supports range queries.

Important note: If the main queries search only by date, a separate index on Transaction_Date may also be needed because the composite index begins with Account_No.

Conclusion: A composite B+ Tree index on (Account_No, Transaction_Date) is suitable for the banking system.

Q3. Healthcare system accessed by PatientID and Doctor Name

Use a multi-level indexing solution with a primary index on Patient_ID and a secondary index on Doctor_Name.

Primary index:

Patient_ID → Record Address

Example:

1001 → 500
 1002 → 620
 1003 → 740
 1004 → 860

Secondary index:

Doctor_Name → Patient_ID

Example:

Dr. Arun → 1001, 1005
 Dr. Bala → 1002, 1006
 Dr. Kumar → 1003, 1007

Multi-level structure:

Top-Level Index

```

  |
+-----+-----+
|       |
Patient ID  Doctor Name
|       |
Second     Second
Level      Level

```

| |
Records Patient IDs

For Patient_ID = 1003, the primary index directly locates the record.

For Doctor_Name = Dr. Kumar, the secondary index finds all associated Patient IDs, which are then used to retrieve records.

Advantages:

- Fast PatientID search.
- Fast doctor-based search.
- Reduced disk I/O.
- Suitable for large healthcare databases.

Conclusion: Use a primary multi-level B+ Tree index on Patient_ID and a secondary multi-level index on Doctor_Name.

Q4. Logistics company – B-Tree or B+ Tree?

For a logistics system with frequent dynamic insertions, deletions and searches, a B+ Tree is more suitable.

Comparison:

B-Tree:

- Data may be stored in internal and leaf nodes.
- Good search performance.
- Good for dynamic updates.

B+ Tree:

- Actual data entries are stored at leaf level.
- Leaf nodes are linked.
- Excellent range queries.
- High fan-out gives a smaller tree height.
- Efficient sequential access.

Example leaf links:

[1001 1002 1003] → [1004 1005 1006] → [1007 1008 1009]

Insertion:

Find leaf → Insert key → Split if necessary.

Deletion:

Find leaf → Delete key → Redistribute or merge if necessary.

Logistics queries often involve ranges, such as finding shipments between dates or shipment IDs.

Conclusion: B+ Tree is more suitable because it maintains balance during dynamic updates and provides efficient range and sequential retrieval.

Q5. Hashing scheme for HR system with 5000 employees

Use Employee_ID as the hashing key.

Hash function:

$$h(\text{Employee_ID}) = \text{Employee_ID} \% M$$

For example, if $M = 100$:

$$h(1250) = 1250 \% 100 = 50$$

Static hashing:

- Fixed number of buckets.
- Simple and fast.
- Average search is $O(1)$.
- Overflow chains may occur.
- Difficult to resize when records grow.

Extendible hashing:

- Uses a directory and dynamic buckets.
- Bucket overflow causes splitting.
- If local depth equals global depth, the directory doubles.
- Handles database growth efficiently.

Example:

Directory:

00 → Bucket

01 → Bucket

10 → Bucket

11 → Bucket

Overflow → Bucket Split → Directory Doubling if required.

Comparison:

Static hashing: simple, fixed size, less suitable for growth.

Extendible hashing: dynamic, scalable, better overflow handling.

Recommendation: For a fixed database of exactly 5000 employees, static hashing is simple. If employee records will grow or change significantly, extendible hashing is preferred.

Q6. Airline reservation – point queries by flight number and range queries by date

Use two complementary indexes.

1. Point query index:

Create a B+ Tree or hash index on Flight_No.

Example:

AI101 → Record Address

AI102 → Record Address

AI103 → Record Address

2. Range query index:

Create a B+ Tree index on Flight_Date.

Example:

Aug 01 → Aug 05 → Aug 10 → Aug 15

Point query:

SELECT * FROM Flight WHERE Flight_No = 'AI101';

Range query:

SELECT * FROM Flight
WHERE Flight_Date BETWEEN '2026-08-01' AND '2026-08-15';

Structure:

Flight Table

+---+-----+	
Flight_No	Flight_Date
B+ Tree/Hash	B+ Tree
Point Search	Range Search

Conclusion: Use a point-oriented index on Flight_No and a B+ Tree range index on Flight_Date.

Q7. University student database – department and CGPA

Use a sorted file organized by Student_ID with secondary indexes on Department and CGPA.

Student fields:

Student_ID, Name, Department, CGPA, Year

Primary file organization:

1001 → Arun

1002 → Bala

1003 → Cathy

1004 → David

Secondary Department index:

CSE → 1001, 1005, 1010

ECE → 1002, 1006

MECH → 1003, 1007

Secondary CGPA index:

Use a B+ Tree on CGPA so that range queries are efficient.

Example query:

SELECT *
FROM Student

WHERE Department = 'CSE'
AND CGPA BETWEEN 8.0 AND 9.0;

Structure:

Student File

```
|
+---+-----+
|           |
Department Index  CGPA Index
|           |
CSE/ECE/MECH      B+ Tree
|           |
Student IDs       CGPA Range
```

Conclusion: Use sorted Student_ID organization, a secondary Department index, and a B+ Tree secondary index on CGPA.

Q8. Disk block utilization for Heap, Sorted and Hashed files

Given:

Number of records = 100,000

Record size = 50 bytes

Block size = 4 KB = 4096 bytes

Records per block:

$\text{floor}(4096 / 50) = 81$ records/block

Number of blocks:

$\text{ceil}(100,000 / 81) = 1235$ blocks

Heap file:

- Approximately 1235 data blocks.
- Records are stored wherever space is available.
- Very fast insertion.
- Sequential scan may require about 1235 block accesses in the worst case.

Sorted file:

- Approximately 1235 data blocks.
- Records are maintained in sorted order.
- Efficient search and range queries.
- Insertions can be more expensive because order must be maintained.

Hashed file:

- Approximately 1235 data blocks before considering overflow.
- Records are distributed among buckets.
- Very efficient equality search.
- Overflow blocks may increase storage.

Comparison:

Heap → Fast insertion.

Sorted → Good search and excellent range queries.

Hashed → Fast equality search.

Conclusion: The basic data storage requirement is approximately 1235 blocks for all three organizations, but actual performance differs because of their access methods and possible overflow/reorganization.

Q9. Social media platform – 500 million posts

For 500 million posts, use composite B+ Tree indexes designed around the main query patterns.

Table:

POST(post_id, user_id, timestamp, hashtag, content)

Index 1:

```
CREATE INDEX idx_user_time  
ON Post(user_id, timestamp);
```

This supports user timeline and user/date-range queries.

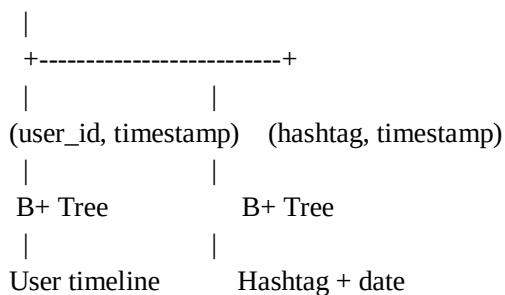
Index 2:

```
CREATE INDEX idx_hash_time  
ON Post(hashtag, timestamp);
```

This supports hashtag and date-range queries.

Structure:

POSTS



For 500 million rows:

- Index only frequently used query patterns.
- Consider partitioning by time for very large data.
- Avoid unnecessary indexes because every index increases insert/update cost.

Column order matters. In (user_id, timestamp), the database first locates the user and then searches the timestamp range.

Conclusion: Use B+ Tree indexes on (user_id, timestamp) and (hashtag, timestamp), together with suitable partitioning for scalability.

Q10. Compare Heap, Sorted and Hashed File for 1 million daily billing transactions

The supermarket billing system performs frequent insertions, transaction searches and updates.

Assume Transaction_ID is the primary key.

1. Heap File:

Insert: Very fast, approximately $O(1)$.

Search: $O(N)$ because a sequential scan may be required.

Delete: Requires locating the record first.

Best for high-volume insertion.

2. Sorted File:

Search: $O(\log N)$ using binary search.

Insert: More expensive because sorted order must be maintained.

Range queries: Excellent.

Suitable when ordered retrieval is important.

3. Hashed File:

Hash function:

$$h(\text{Transaction_ID}) = \text{Transaction_ID} \% M$$

Average search: $O(1)$

Average insert: $O(1)$

Range queries: Poor because hashing does not preserve order.

Comparison:

Operation	Heap	Sorted	Hashed
Insert	Excellent	Moderate	Excellent
Search	Poor	Good	Excellent
Delete	Moderate	Moderate	Good
Range Search	Poor	Excellent	Poor
Traversal	Poor	Excellent	Poor

Recommended practical design:

Transaction_ID $\rightarrow$ Hash/B+ Tree Index

Transaction_Date $\rightarrow$ B+ Tree Index

This gives fast transaction-ID lookup, fast insertion, and efficient date-based reports.

Conclusion: Hashing is best for exact transaction-ID searches, while B+ Tree indexing is better for date/range queries. A combined design provides strong practical performance.

Quick Revision Table

Q.No	Topic	Recommended Concept
1	10 million products	B+ Tree + sorted/clustered organization
2	Bank transactions	Composite (Account_No,

		Transaction_Date) B+ Tree
3	Healthcare records	Primary + secondary multi-level indexing
4	Logistics	B+ Tree
5	5000 employees	Extendible hashing for dynamic growth
6	Airline reservations	Flight_No point index + Flight_Date B+ Tree
7	University database	Sorted Student_ID + Department + CGPA indexes
8	100,000 records	81 records/block; 1235 blocks
9	500 million posts	(user_id, timestamp) + (hashtag, timestamp)
10	Supermarket billing	Hashing for equality + B+ Tree for ranges

Important Formulas

Records per block = $\text{floor}(\text{Block size} / \text{Record size})$

Number of blocks = $\text{ceil}(\text{Number of records} / \text{Records per block})$

Hash function: $h(\text{key}) = \text{key} \% M$

Binary/B+ Tree search: $O(\log N)$

Hash search – average: $O(1)$

Heap sequential search: $O(N)$

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution	25%	Innovative,	Logical and	Basic	Unclear or
Design		practical, and feasible solution aligned with industry standards	feasible solution with minor gaps	solution with limited feasibility	impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation